

Agent Phantom Recovery

07-trd.md

Technical Requirements Document (TRD)

Part 4

Performance Targets, Deployment, Acceptance Criteria & Future Expansion

26. Performance Requirements

26.1 Performance Philosophy

The platform should optimize for:

1. Correctness
2. Reliability
3. Traceability
4. Completion
5. Speed

Fast incorrect results are less valuable than slower verified results.

26.2 Response Performance

User Interface Actions

Target:

< 200 ms

Examples:

- panel switching
- sidebar interaction
- workspace navigation
- settings changes

Task Initialization

Target:

< 5 seconds

Includes:

- task creation
- plan generation
- context assembly

Repository Indexing

Small Repository:

< 30 seconds

Medium Repository:

< 2 minutes

Large Repository:

< 10 minutes

Initial indexing may take longer than subsequent runs due to caching.

Memory Retrieval

Target:

< 2 seconds

Includes:

- project memory
- session memory
- experience memory

26.3 Tool Performance

Terminal Execution

The platform should stream results in real time.

Users should not wait for process completion before seeing output.

Browser Actions

Target:

< 3 seconds

for common interactions such as:

- navigation
 - refresh
 - inspection
-

Search Retrieval

Target:

< 2 seconds

for indexed repositories.

26.4 Long-Horizon Performance

The system shall support:

- extended sessions
- large repositories
- long-running investigations

without significant degradation in reasoning quality.

27. Resource Management Requirements

27.1 Compute Efficiency

The platform shall minimize unnecessary:

- model calls
- tool invocations
- context assembly
- memory retrieval

Reasoning should occur only when required.

27.2 Context Efficiency

Large repositories should be narrowed before reasoning.

Preferred workflow:

```
Repository
↓
Index
↓
Rank
↓
Candidate Extraction
↓
Reasoning
```

The system must avoid sending entire repositories to models whenever possible.

27.3 Token Efficiency

The architecture should support:

- summarization
- chunking
- ranking
- retrieval

to reduce model costs.

28. Deployment Requirements

28.1 Deployment Philosophy

The platform shall support production deployment from the beginning.

Development-only architecture should be avoided.

28.2 Environment Separation

The platform shall support:

Development

Used for:

- local development
 - testing
 - debugging
-

Staging

Used for:

- integration testing
 - performance validation
 - release verification
-

Production

Used for:

- real users
 - persistent workloads
 - operational deployment
-

28.3 Deployment Model

Preferred deployment:

```
Frontend
↓
API Gateway
```

↓
Backend Services
↓
Storage Services

Each major service should be independently deployable.

28.4 Containerization

All backend services should support:

- containerized deployment
- environment isolation
- reproducible builds

Recommended approach:

Docker
+
Container Orchestration

28.5 Configuration Management

Configuration should be externalized.

Examples:

- API keys
- model settings
- database connections
- feature flags

Configuration must not be hardcoded.

29. Security Operations Requirements

29.1 Security Philosophy

Security must be built into infrastructure rather than added later.

29.2 Authentication

The platform shall support:

- user authentication
 - session validation
 - token refresh
 - logout
-

29.3 Authorization

The system shall enforce permissions for:

- repositories
 - tools
 - memory access
 - execution actions
-

29.4 Audit Logging

Critical actions shall be logged.

Examples:

- repository access
 - file modifications
 - tool execution
 - task creation
 - memory updates
-

29.5 Secret Management

Secrets shall be stored securely.

Examples:

- API keys
- provider credentials
- access tokens

Secrets must never be exposed to users.

29.6 Secure Execution

Execution environments should be isolated.

Goals:

- containment
 - traceability
 - stability
-

30. Cost Management Requirements

30.1 Cost Philosophy

The platform should be capable of scaling without uncontrolled cost growth.

30.2 Model Cost Optimization

The architecture should support:

Planner

Lower-cost model.

Reasoner

Higher-capability model.

Verifier

Balanced model.

This allows intelligent allocation of compute resources.

30.3 Tool Cost Optimization

The platform should avoid:

- duplicate scans
- duplicate searches
- duplicate indexing

Results should be cached where appropriate.

30.4 Storage Cost Optimization

Support:

- archival
- retention policies
- deduplication

to prevent unnecessary storage growth.

31. Production Readiness Requirements

31.1 Production Standards

The system should not be considered production-ready until it supports:

- monitoring
 - logging
 - recovery
 - testing
 - deployment automation
-

31.2 Operational Visibility

Operators should be able to determine:

- current system health
- running tasks
- failed tasks
- resource consumption

without accessing source code.

31.3 Recovery Readiness

The system must recover gracefully from:

- process crashes
 - provider outages
 - network failures
 - execution interruptions
-

32. Technical Acceptance Criteria

TAC-1 Repository Investigation

The platform shall successfully:

- ingest repositories
 - understand structure
 - identify important regions
-

TAC-2 Planning

The platform shall generate executable plans.

TAC-3 Tool Usage

The platform shall:

- invoke tools
- interpret outputs
- continue execution

without manual intervention after every step.

TAC-4 Memory

The platform shall maintain context throughout long tasks.

TAC-5 Verification

The platform shall verify conclusions before final delivery.

TAC-6 Patch Generation

The platform shall generate valid code modifications.

TAC-7 Recovery

The platform shall recover from failed strategies through replanning.

TAC-8 Deliverables

The platform shall produce:

- reports
 - patches
 - summaries
 - verification records
-

33. MVP Exit Criteria

Version 1 shall not be considered complete until all of the following are functional.

Core Agent

- Goal intake
 - Planning
 - Replanning
 - State management
-

Tooling

- Terminal
 - Browser
 - Search
 - File System
 - Git Integration
-

Memory

- Working memory
 - Session memory
 - Project memory
-

Verification

- Verification workflow
 - Patch review
 - Test validation
-

Workspace

- Browser panel
 - Source code panel
 - Terminal panel
 - Agent console
 - Sidebar
-

Deliverables

- Reports
 - Patches
 - Task summaries
-

34. Future Expansion Requirements

34.1 Advanced Memory

Future versions may support:

- hierarchical memory
 - memory ranking models
 - autonomous memory maintenance
-

34.2 Multi-Agent Collaboration

Future versions may support:

```
Planner Agent
↓
Research Agent
↓
Reasoning Agent
↓
Verification Agent
↓
Delivery Agent
```

under a central orchestrator.

34.3 Advanced Repository Intelligence

Future enhancements may include:

- architecture reconstruction
 - dependency visualization
 - repository knowledge graphs
-

34.4 Advanced Verification

Future verification systems may include:

- automated regression analysis
 - formal validation
 - enhanced test generation
-

34.5 Enterprise Features

Potential future additions:

- team workspaces
 - organization memory
 - role-based access control
 - compliance reporting
-

35. Architectural Decision Summary

The platform is intentionally designed around:

Long-Horizon Execution

```
Goal
↓
Plan
↓
Execute
↓
Observe
↓
Replan
↓
Verify
```

↓
Deliver

Three-Model Architecture

Planner
↓
Reasoner
↓
Verifier

Tool-First Intelligence

Evidence
↓
Reasoning
↓
Verification

Memory-Centered Operation

Working Memory
+
Session Memory
+
Project Memory
+
Experience Memory

Execution-Focused UX

Browser
+
Source Code
+
Terminal
+
Agent Console

rather than a chat-only interface.

36. Final System Definition

Agent Phantom Recovery is a long-horizon autonomous engineering and security recovery platform that combines:

LLMs
+
Agents
+
Tools
+
Memory
+
Planning
+
Verification

to investigate, understand, repair, validate, and complete complex technical work.

The system is designed to function as an execution-oriented engineering platform rather than a conversational assistant.

Its defining characteristic is not the ability to generate answers.

Its defining characteristic is the ability to complete work.

End of Technical Requirements Document (TRD)

Status: Complete Version: 1.0 Draft Document Type: Production Technical Requirements Document